

Training Model 1, start to finish

A complete walkthrough for someone who has never trained a model before — from an empty folder to weights the sorting machine can use.

TRAINS

Model 1 only

ARCHITECTURE

YOLOv8n-cls

CLASSES

6 varieties

IMAGES NEEDED

~3,000

FRUITS TO SHOOT

~600

TIMELINE

Weeks, then an afternoon

Contents

ORIENTATION

- Read this part before anything else

PHASE 1 • SET UP AND LEARN THE ONE RULE

- 1** Confirm the code runs on your machine
- 2** Learn the fruit id rule

PHASE 2 • COLLECT THE IMAGES

- 3** Set up the shoot
- 4** Name the files and cull the bad frames
- 5** Verify the split — after every class

PHASE 3 • TRAIN

- 6** Upload the images and the code to Drive
- 7** Open Colab and fix the broken cell
- 8** Run the cells
- 9** Read the results honestly

PHASE 4 • CONNECT IT TO MODEL 5

- 10** Produce a prediction and route it

REFERENCE

- A** If Colab is unavailable
- B** Troubleshooting
- C** Vocabulary
- D** The knobs you can turn

Read this part before anything else

Three facts shape everything below. If you skip them, the later steps will look arbitrary.

1. Only one of your two models trains

`MyTasks/` contains two projects. **Model 5** (`model5_fusion/`) is a *rule engine* — it reads the outputs of Models 1–4 and applies five ordered rules to pick a bin. It has no weights, no dataset, and nothing to train. It already works; I ran its suite and all 284 tests pass.

Model 1 (`model1_variety/`) is the one that learns. It looks at photographs of a mango and names the variety. That is what this handbook is about.

2. You have a finished pipeline and zero data

Every script you need is written and tested — I ran Model 1's core suite too, 54 tests passing in 0.10 seconds on your Python 3.14.4. What is missing is images. There is no public dataset for any of your six varieties (the good mango datasets are all Bangladeshi and Pakistani cultivars with zero class overlap), so you are photographing roughly 600 mangoes yourself.

That is why the timeline reads "weeks, then an afternoon." Collecting is the whole job. Training is a notebook you run once.

3. There is a broken step in the existing instructions

KNOWN BLOCKER • FIXED IN STEP 7

The Colab notebook's third cell runs `git clone https://github.com/mangoscan-a11y/MangoScan.git` and then changes into `MyTasks/model1_variety`. That will fail.

Line 28 of your `.gitignore` is `/MyTasks`, and `git ls-files MyTasks` returns nothing — the training code was never committed, so it is not in the repo the notebook clones. Step 7 replaces that cell.

Phase 1 - Set up and learn the one rule ONE EVENING

Do this before you touch a camera. Step 2 in particular — it is the difference between a real result and a thesis chapter built on a number that means nothing.

1 Confirm the code runs on your machine

10 MINUTES · YOUR LAPTOP · NO INSTALLS

Open PowerShell. In Windows, press `Win`, type *PowerShell*, press Enter. Then paste these two lines:

```
cd "C:\Users\ANGELO GALLEROS\GitHub\MangoScan\MyTasks\model1_variety"
python -m pytest -q
```

You should see:

```
..... [100%]
54 passed in 0.10s
```

This proves the parts that matter — the splitting logic, the five-view averaging, the metrics — work on your Python. None of it needs a GPU or an install, which is deliberate: the project is split so everything worth testing runs locally and only the GPU work needs Colab.

CHECKPOINT

54 tests pass. If Python is not found at all, install Python from python.org and reopen PowerShell.

2 Learn the fruit id rule

15 MINUTES · READ TWICE

This is the single most important idea in the project, so here it is from first principles.

WHY DATA GETS SPLIT INTO THREE PILES

A model learns by looking at examples. To find out whether it actually learned anything — rather than memorising the specific photos it saw — you hide some images from it and test on those. Your config splits **70% train / 15% validation / 15% test**.

- **Train** — the model studies these.
- **Validation** — checked after every pass, to decide when to stop.
- **Test** — touched once, at the very end. Your real score.

WHY SPLITTING BY IMAGE IS WRONG HERE

You photograph each mango **five times** from five angles. Those five photos are nearly identical. If the split happens per *image*, views 1, 2 and 4 of one mango land in train while view 3 lands in test — and the model is graded on a mango it already studied. Your accuracy climbs, your real accuracy does not, and **every metric you would normally check still looks perfectly fine**.

So the pipeline splits by *physical fruit* instead. All five views of one mango always travel together. It works this out from the filename.

THE NAMING CONVENTION

```
data/raw/<Class_Name>/<class>_<fruit_id>_v<view>.jpg
```

```
data/raw/Carabao/carabao_001_v1.jpg ← fruit 001, view 1
data/raw/Carabao/carabao_001_v2.jpg ← the SAME physical mango
data/raw/Carabao/carabao_001_v5.jpg ← still the same mango
data/raw/Carabao/carabao_002_v1.jpg ← a DIFFERENT mango
data/raw/Apple_Mango/apple_mango_014_v3.jpg
```

The pipeline strips the `_v<n>` off the end and treats whatever remains as the fruit id. `carabao_001_v1` and `carabao_001_v5` both reduce to `carabao_001`, so they are one mango.

THE TWO WAYS TO GET THIS WRONG

Two different mangoes sharing an id — they get treated as one fruit and always land in the same pile. Mild loss of precision, survivable.

One mango's views getting different ids — its views scatter across train and test. This is the failure that silently invalidates a whole thesis chapter, and no metric will reveal it.

THE SIX DIRECTORY NAMES, SPELLED EXACTLY

Underscores in folder names, spaces in the labels. These map to `mango_varieties.variety_name` in Supabase, so a typo here becomes a database error much later.

```
data/raw/Carabao/
data/raw/Apple_Mango/
data/raw/Indian/
data/raw/Chupadera/
data/raw/Wani/      ← "Wani", not "Wanni"
data/raw/Kabayo/
```

WHERE TO PUT THE FOLDER

Make `data/raw/` inside `model1_variety/`, or anywhere else convenient — `configs/dataset.toml` has a `raw_dir` setting you can point wherever you like. Keep it off a synced drive while you are working; sync tools rename files.

Phase 2 · Collect the images

THE CRITICAL PATH · WEEKS

Work one variety at a time and finish it completely — shoot, name, verify — before starting the next. Discovering a naming problem after all six classes are shot means re-shooting from memory, which you cannot do.

3 Set up the shoot

HALF A DAY, ONCE · THE RIG

The governing principle: **your training images should be indistinguishable from what the machine will see when it is running**. Every difference is a gap the model has to bridge, and small datasets bridge gaps badly.

USE THE MACHINE'S OWN RIG

Shoot on the sorting machine itself if it is assembled — its cameras, its lights, its conveyor surface. This one decision is worth more than doubling your image count. If the rig is not ready, replicate it as closely as you can and keep every setting **identical across all six classes**. A setup that drifts between classes teaches the model to recognise the setup rather than the mango.

THE FIVE ANGLES

VIEW	ANGLE
v1	Stem end (peduncle)
v2	Blossom end
v3	Cheek A — the flatter face
v4	Cheek B — rotate 180° from v3
v5	Top-down

If the rig uses different angles, **the rig wins**. What matters is that the same five angles are used for every mango in every class. Consistency beats any particular choice.

LIGHTING AND BACKGROUND

- Turn on the machine's lighting and leave it alone. Do not adjust exposure or white balance between classes.
- **Never mix a phone flash with rig lighting inside one class.** Flash changes how the skin reflects, and skin appearance is exactly the signal the model is learning.
- If ambient light leaks into the enclosure, shoot every class at the same time of day, or block the leak.
- Background is the conveyor surface, always. A class shot on a different surface is the classic way to get a model that scores 99% in validation and fails on the line — it learned the background.

HOW MANY, AND HOW VARIED

PER CLASS	FRUITS	IMAGES
Absolute minimum	60	300
Target	100+	500+
All six classes at target	~600	~3,000

Vary ripeness across the green-to-yellow range the machine will actually see, vary size across small/medium/large, and use different trees and harvest days where you can.

FRUIT COUNT IS WHAT COUNTS

Photographing the same 10 mangoes 30 times each gives you 300 images and a model that has seen ten mangoes. The pipeline counts fruits, and so should you.

NUMBER THE MANGO BEFORE YOU PHOTOGRAPH IT

Write the id on your label sheet, *then* shoot. Numbering afterwards from memory is exactly how ids get reused and mangoes get merged. Copy `tools/label_sheet_template.csv` and fill a row per fruit as you go:

```
fruit_id,variety,date_captured,views_captured,ripeness_note,size_note,defect_note,captured_by
carabao_001,Carabao,2026-08-23,5,green,medium,none,
carabao_002,Carabao,2026-08-23,5,yellow,large,minor bruise,
```

Ids only need to be unique *within* their class — `carabao_001` and `wani_001` coexist fine.

4

Name the files and cull the bad frames

AS YOU SHOOT · POWERSHELL

Cameras produce names like `DSC_0431.JPG`. Renaming three thousand of those at the end is miserable and error-prone, so rename after each mango instead — five files at a time, while you still know which mango it was.

A RENAME HELPER

Shoot v1 through v5 in order, then run this once. It takes the five newest unnamed files in the folder and names them for that mango:

```
# Edit these three lines for each mango, then run.
$class = "carabao"
$id     = "001"
$dir    = "C:\MangoScan\data\raw\Carabao"

$files = Get-ChildItem $dir -File |
    Where-Object { $_.Name -notmatch "_v\d+\." } |
    Sort-Object LastWriteTime | Select-Object -Last 5
if ($files.Count -ne 5) { Write-Warning "Found $($files.Count) files, expected 5" }
$i = 1
foreach ($f in $files) {
    Rename-Item $f.FullName ("{0}_{1}_v{2}{3}" -f $class, $id, $i, $f.Extension.ToLower())
    $i++
}
```

It sorts by modification time, so view numbers follow your shooting order — which is why shooting v1→v5 in sequence matters. The `-notmatch "_v\d+\."` filter skips files you already renamed, so running it twice will not clobber earlier mangoes.

TEST IT ON THREE MANGOES FIRST

Run it, look at the folder, confirm the names are what you expect. Only then trust it for the rest of the shoot.

DELETE BAD FRAMES — DO NOT TRY TO FIX THEM

Throw away any frame that is:

- Blurred or motion-smeared
- Showing two mangoes
- Cut off at the edge of the image
- Obscured by a hand or tool

Dropping a frame is genuinely free — Model 1 averages however many views it gets and handles fewer than five gracefully. Keeping a bad frame is not free: in training it is noise, and in test it is a wrong answer you will spend an afternoon investigating. Note the drop in the label sheet's `views_captured` column so your counts still reconcile.

5

Verify the split — after every class

2 MINUTES · YOUR LAPTOP · REPEAT 6×

This is the checkpoint that catches the leak while it is still cheap to fix. The preparation script is stdlib-only, so it runs on your Python 3.14 today — no Colab needed. First point the config at your images by editing `configs/dataset.toml`:

```
[paths]
raw_dir = "C:/MangoScan/data/raw"    ← forward slashes are fine
```

Then run it with `--dry-run`, which copies nothing and only reports:

```
cd "C:\Users\ANGELO GALLEROS\GitHub\MangoScan\MyTasks\model1_variety"
python scripts/01_prepare_dataset.py --config configs/dataset.toml --dry-run
```

Healthy output looks like this:

```
Carabao          312 images    63 fruits  train/val/test = 44/9/10
Apple Mango      295 images    59 fruits  train/val/test = 41/9/9
...

train           1247 images
val              267 images
test            271 images
```

CHECKPOINT — CHECK THE FRUITS COLUMN

63 fruits should equal the 63 rows on your label sheet. That one comparison is the entire point of this step.

WHAT YOU SEE

WHAT IT MEANS

312 images
312 fruits

This is the leak. The `_v<n>` suffix is missing or malformed, so every image is being treated as its own mango. Fix the filenames before doing anything else.

fruits *more than* label sheet rows

An id got mistyped, splitting one mango into two.

fruits *fewer than* label sheet rows

Two mangoes were given the same id.

only N distinct fruits
- need at least 3

Same root cause as the leak, or you genuinely have too few fruits yet.

no images in
data/raw/<X>

Directory name is wrong. Underscores, not spaces: `Apple_Mango`.

PRE-FLIGHT CHECKLIST, BEFORE YOU UPLOAD ANYTHING

- ☐ Six directories exist, spelled exactly right
- ☐ Every file matches `<class>_<fruit_id>_v<view>.<ext>`

- ☐ Each fruit id is unique within its class and shared by all its views
- ☐ At least 60 distinct fruits per class (300+ images)
- ☐ Blurred, double-mango and cut-off frames deleted
- ☐ Label sheet filled in and reconciled against the dry run
- ☐ The dry run exits with no **Problems:** section

Phase 3 • Train ONE AFTERNOON

Training happens on Google Colab — a free service that runs Python notebooks on Google's GPUs. Not because your RTX 3050 is too weak, but because PyTorch has no builds for Python 3.14 yet, so `pip install ultralytics` simply fails on your machine.

6 Upload the images *and* the code to Drive

HOURS OF UPLOAD • START IT AND WALK AWAY

Colab cannot see your laptop. Google Drive is how you hand it both the dataset and the scripts. Sign in to `drive.google.com` with the same account you will use for Colab, and build this structure:

```
MyDrive/MangoScan/  
  data/raw/Carabao/carabao_001_v1.jpg  
  data/raw/Apple_Mango/...  
  data/raw/Indian/...  
  data/raw/Chupadera/...  
  data/raw/Wani/...  
  data/raw/Kabayo/...  
  code/model1_variety/      ← the whole project folder
```

Upload the folder, not a zip. The notebook reads the files directly and there is no unzip step. Drag `raw` into the browser window and leave it running — a few thousand JPEGs takes a while.

For the code, drag `MyTasks/model1_variety` into `MangoScan/code/`. Skip the `.pytest_cache` and `__pycache__` folders if your uploader lets you; they are just caches.

WHY THE CODE GOES TO DRIVE AT ALL

Because of the blocker from the opening section: `MyTasks` is gitignored and was never pushed, so Colab cannot clone it. Uploading it to Drive sidesteps that without changing your repo.

The alternative is to remove `/MyTasks` from `.gitignore`, commit and push, and keep the notebook's original clone cell. Reasonable if the folder was ignored by accident — but if it was ignored on purpose, stay with Drive.

7 Open Colab and fix the broken cell

10 MINUTES • BROWSER

OPENING THE NOTEBOOK

1. Go to `colab.research.google.com` and sign in.
2. **File** → **Upload notebook**, and pick `MyTasks/model1_variety/notebooks/MangoScan_Model1_Colab.ipynb` from your laptop.
3. **Runtime** → **Change runtime type** → **T4 GPU**, then Save.

DO NOT SKIP THE T4 STEP

Miss it and training silently falls back to the CPU, turning a 30-minute job into a multi-hour one. A notebook is a list of cells you run top to bottom with the ► button or `Shift + Enter`; the GPU is what those cells run on.

REPLACE CELL 3

Find the cell containing `!git clone`. Delete everything in it and paste this instead:

```
!pip install -q ultralytics

import shutil, pathlib
SRC = pathlib.Path('/content/drive/MyDrive/MangoScan/code/model1_variety')
assert SRC.is_dir(), f'Upload model1_variety to {SRC} first'
shutil.copytree(SRC, '/content/model1_variety', dirs_exist_ok=True)
%cd /content/model1_variety
!ls
```

It copies the code out of Drive into Colab's local disk before running. That matters: training writes to `runs/` constantly, and Drive is slow enough that working in it directly would bottleneck the GPU.

8

Run the cells

20–50 MINUTES • MOSTLY WAITING

Top to bottom, one at a time, waiting for each to finish.

CELL	WHAT IT DOES	TIME
1	<code>nvidia-smi</code> — prints the GPU. If it errors, you missed the T4 step.	seconds
2	Mounts Drive (a permission popup appears — accept it) and lists your class folders with file counts. Check these against your label sheet.	a minute
3	Installs Ultralytics and loads your code — the cell you just replaced.	1–2 min
5	Rewrites <code>dataset.toml</code> to point at Drive. This edit is local to Colab and does not touch your repo.	instant
6	<code>01_prepare_dataset.py</code> — the same fruit-grouped split you dry-ran, this time for real.	< 1 min
8	<code>02_train.py</code> — the actual training.	15–40 min
10–11	<code>03_evaluate.py</code> plus the confusion matrix plot.	1–2 min
13	<code>04_export.py</code> — ONNX conversion and a parity check.	1–2 min
15	Copies <code>runs/</code> back to Drive so nothing is lost when the session ends.	1 min

WHAT YOU ARE WATCHING DURING CELL 8

The model makes 100 passes over your images — each pass is an **epoch**. After every one it scores itself on the validation pile and prints `val/top1_acc`, the fraction it got right. You want that number climbing and then flattening. Training stops early if 20 epochs pass with no improvement, and keeps the best version rather than the last.

STOP IF THIS HAPPENS

`val/top1_acc` reaching about **1.00 in the first epoch is not success**. It means the model is being tested on images it just studied — a split leak. Kill the run and go back to Step 5. Nothing you do later can repair this.

CHECKPOINT

Cell 8 ends by printing `Best weights: runs/model1_variety/weights/best.pt`. That file is your trained model.

Read the results honestly

30 MINUTES · THE PART THAT GOES IN YOUR PAPER

`runs/model1_variety/weights/report.json` holds everything, and cell 10 prints the headline numbers. Three of them decide whether the run was real.

`multiview.top1_accuracy`

The headline. Accuracy per *mango*, after averaging its five views. This is what the machine actually achieves, so it is the number to report.

The averaging is mean-of-softmax rather than a majority vote across the five views, which is why one blurred view among four sharp ones degrades the answer gently instead of casting a full wrong vote.

`multiview_gain`

Multi-view accuracy minus single-image accuracy: how much five cameras buy you over one. A genuine result worth reporting on its own.

IT MUST BE POSITIVE

A negative gain almost always means views of *different* mangoes are being averaged together — the fruit ids are wrong. Back to Step 2, then Step 5.

`multiview.ece` — **Expected Calibration Error**

The gap between the confidence the model claims and the accuracy it delivers. A model that says "90% sure" should be right about 90% of the time; ECE measures how far off that promise is.

This matters more here than in an ordinary classifier, because **Model 5 gates routing on Model 1's confidence**. An overconfident model does not merely report a wrong number — it pushes bad reads through the gate and into the wrong bin.

ECE	READING
< 0.05	Good. Model 5's default 0.60 thresholds are sound.
0.05 – 0.15	Acceptable. Keep an eye on it.
> 0.15	Overconfident. Retune <code>thresholds.variety</code> in <code>model5_fusion/config/routing.toml</code> against this specific model, or fit temperature scaling on the validation split.

`reliability.csv` and the plotted reliability diagram show *where* the miscalibration sits — usually in the top confidence bin.

Also worth reading

- `per_class` **precision / recall / F1** — finds the one variety dragging the average down.
- `macro_avg` vs `weighted_avg` — a big gap between them means your classes are imbalanced.
- `top3_accuracy` — if top-1 is poor but top-3 is high, the model is confusing a specific *pair* of varieties. The confusion matrix from cell 11 shows which pair, and that usually points at a real

visual similarity you can photograph your way out of.

FILES PRODUCED

FILE	CONTENTS
best.pt	The trained weights. This is the model.
best.onnx	Portable export for deployment.
report.json	Every metric described above.
confusion_matrix.csv	What got mistaken for what.
reliability.csv	Confidence vs. actual accuracy, per bin.

10 Produce a prediction and route it

PROVES THE LOOP CLOSES

Your weights are now at `MyDrive/MangoScan/runs/model1_variety/weights/best.pt`. Turn five images of one mango into the JSON payload Model 5 expects:

```
python scripts/05_predict_multiview.py \  
  --weights best.pt \  
  --images v1.jpg v2.jpg v3.jpg v4.jpg v5.jpg \  
  --scan-id scan-0001 --declared-variety Carabao > scan.json
```

Then feed it to the fusion engine, which runs on your laptop with no installs:

```
cd ../model5_fusion  
python -m fusion.cli --explain - < ../model1_variety/scan.json
```

You get the bin and the rule trace that chose it:

```
--- scan-0001 -> bin 4 (YELLOW_SMALL)  
  confidence_gate  pass      all routing dimensions trusted  
  disease          pass      Healthy  
  bruise           pass      not bruised  
  grade            bin 4      Yellow x Small  
  variety_check    match      Carabao
```

VARIETY NEVER ROUTES

Mangoes are pre-sorted by variety before entering the machine, so Model 1's job is to *verify* that pre-sort. A disagreement with the operator's declared batch variety raises `VARIETY_MISMATCH` and the mango routes normally on colour and size. Model 1 is quality control, not the router.

Until Models 2–4 exist, the mock models stand in for them. This works right now, before you have collected a single image:

```
cd MyTasks\model5_fusion  
python -m stubs.mock_models --count 20 | python -m fusion.cli --explain -
```

Plenty will land in bin 7 with `LOW_CONFIDENCE`. That is the mock doing its job, not a bug — it draws confidences uniformly from 0.55 to 0.99 against a 0.60 threshold, deliberately exercising the gate. Real models sit well above it.

A If Colab is unavailable

Install Python 3.12 alongside your 3.14 — they coexist fine, and 3.12 has the PyTorch builds 3.14 lacks. Download it from python.org, then:

```
cd "C:\Users\ANGELO GALLEROS\GitHub\MangoScan\MyTasks\model1_variety"
py -3.12 -m venv .venv312
.venv312\Scripts\activate
pip install -r requirements.txt
python scripts/01_prepare_dataset.py --config configs/dataset.toml
python scripts/02_train.py --config configs/dataset.toml --batch 16
```

--BATCH 16 IS NOT OPTIONAL

The config default of 64 is sized for a T4's 15 GB and will run your 4 GB card out of memory immediately. Drop to 8 if 16 still fails. Expect training to take several times longer than Colab.

B Troubleshooting

SYMPTOM	CAUSE AND FIX
CUDA out of memory	Lower <code>--batch</code> — try 16, then 8. Restart the runtime first; a dead process can still be holding VRAM.
no images in data/raw/<X>	Directory name wrong. Underscores, not spaces: <code>Apple_Mango</code> . And it is <code>Wani</code> , not <code>Wanni</code> .
only N distinct fruits	The <code>_v<n></code> suffix is missing or malformed, so grouping failed.
fruits count equals images count	Same cause. This is the leak. Fix before training.
Val accuracy $\approx$ 1.00 in epoch 1	Near-certain split leak. Do not celebrate — re-check fruit ids.
<code>multiview_gain</code> is negative	Views of different mangoes are being averaged. Fruit ids are wrong.
<code>expected classes [...]</code> , <code>found [...]</code>	A class directory is missing or misspelled in the prepared dataset.
Training starts but crawls	Runtime is on CPU. Runtime → Change runtime type → T4 GPU.
Parity check reports mismatches	The ONNX export diverges from the PyTorch model. Re-export; if it persists, deploy the <code>.pt</code> and raise it.
<code>%cd</code> fails in cell 3	You are running the original clone cell. See Step 7.
Colab disconnects mid-training	Free tier has idle limits. Keep the tab visible; cell 15 saves to Drive, so re-run from cell 3 if you lose the session.

c Vocabulary

Class	One of the six varieties the model can output. Fixed order in <code>mango_variety/classes.py</code> — reordering that tuple invalidates every existing checkpoint.
Fruit id	The part of a filename before <code>_v<n></code> . Identifies one physical mango across its five photos.
Split	Dividing images into train / validation / test. Yours is 70/15/15, split by fruit rather than by image.
Leakage	Test images the model already saw in training. Inflates your score without improving the model, and hides from every metric.
Epoch	One full pass over the training images. You run up to 100.
Batch size	How many images go through the GPU at once. Bigger is faster but needs more VRAM. 64 on a T4, 16 on your laptop.
VRAM	Memory on the graphics card. Your RTX 3050 has 4 GB; a Colab T4 has 15 GB.
Weights / .pt	The learned parameters — the trained model as a file. <code>best.pt</code> is the best-scoring epoch, not the last.
Top-1 accuracy	How often the model's first guess is right.
Top-3 accuracy	How often the right answer is in its first three guesses.
Confusion matrix	A grid of true class against predicted class. Shows exactly which varieties get mixed up.
Softmax / confidence	The model outputs six numbers summing to 1 — how strongly it believes each variety. The largest is its confidence.
Calibration / ECE	Whether stated confidence matches real accuracy. Critical here because Model 5 gates on it.
Augmentation	Randomly flipping and rotating training images so the model generalises. Your config keeps hue jitter deliberately low — skin colour carries real variety signal.
ONNX	A portable model format that runs without PyTorch. The parity check confirms it agrees with the original.
venv	An isolated Python installation for one project, so its packages cannot break your system Python.
Runtime	The machine Colab gives you. Free, temporary, and wiped when the session ends — which is why cell 15 copies results to Drive.

D

The knobs you can turn

Everything below lives in `configs/dataset.toml`. You should not need to change any of it, but knowing what is there helps when something misbehaves.

SETTING	DEFAULT	MEANING
<code>model</code>	<code>yolov8n-cls.pt</code>	The "nano" classifier — smallest and fastest, right for a small dataset on embedded hardware.
<code>imgsz</code>	224	Images are resized to 224×224 before training.
<code>epochs</code>	100	Maximum passes over the data.
<code>patience</code>	20	Stop early after this many epochs without improvement.
<code>batch</code>	64	Sized for a T4. Use 16 locally.
<code>seed</code>	42	Makes the split reproducible — same seed, same three piles.
<code>ratios</code>	<code>[0.7, 0.15, 0.15]</code>	Train / validation / test proportions.
<code>degrees</code>	15.0	Random rotation range during augmentation.
<code>flipud</code>	0.0	No vertical flips — mangoes are photographed upright on the rig.
<code>hsv_h</code>	0.015	Hue jitter, kept low on purpose. Skin colour is real signal here.

Built from the code and documents in MyTasks/ as they stand on 2026-08-25 – docs/01-dataset-sourcing-guide.md, docs/02-model1-training-guide.md, tools/capture_protocol.md, the five pipeline scripts, configs/dataset.toml and the Colab notebook. Test counts and the gitignore blocker were verified by running them.